

How Authentication Works

ToDOO project — beginner guide · React + Express + JWT

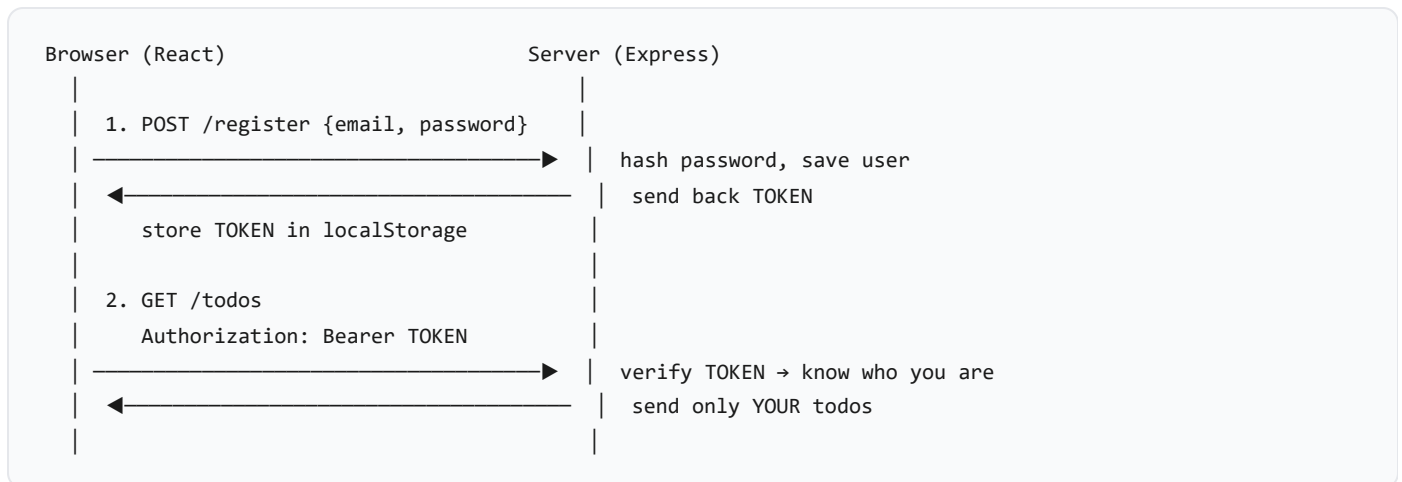
The whole idea in one sentence: the server checks your password *once*, hands you a **token** (a signed pass), and from then on you show that token on every request instead of your password.

1. Why we need auth

Without auth, *everyone* shares the same todo list — anyone can read or delete anyone's todos. Auth lets the server answer two questions on every request:

- **Who are you?** (authentication — proving identity)
- **What may you touch?** (authorization — here: only *your* todos)

2. The big picture



A **token** = a long string the server *signed* with a secret. It says "this is user #123, valid until 2pm". The browser can't fake it because it doesn't know the secret — only the server can create or verify valid tokens.

3. The three core skills

SKILL 1 Hash the password — never store it raw

If the database leaks, raw passwords = disaster. We store a one-way *hash* instead. You can't reverse a hash back to the password.

```
// register
const passwordHash = await bcrypt.hash(password, 10) // save THIS, not the password
```

```
// login
const ok = await bcrypt.compare(password, user.passwordHash) // true / false
```

SKILL 2 Make a token (JWT)

After a correct password, the server signs a token carrying the user's id.

```
const token = jwt.sign({ userId: user.id }, JWT_SECRET, { expiresIn: '2h' })
```

The browser stores it and sends it back as a header on future requests: `Authorization: Bearer <token>`

SKILL 3 Middleware — the gatekeeper

A function that runs *before* protected routes. It reads the token, verifies it, and attaches the user id to the request. Bad/missing token → `401` and the real handler never runs.

```
function authMiddleware(req, res, next) {
  const token = (req.headers.authorization || '').replace('Bearer ', '')
  try {
    const payload = jwt.verify(token, JWT_SECRET) // throws if invalid/expired
    req.userId = payload.userId // now we know WHO
    next() // let the request through
  } catch {
    res.status(401).json({ error: 'Invalid or expired token' })
  }
}
```

4. How it maps to THIS project

File	Role
<code>server/index.js</code>	Backend. Has <code>/register</code> , <code>/login</code> , <code>authMiddleware</code> , and the todo routes (now protected).
<code>src/auth.js</code>	Tiny helpers: save / read / clear the token in <code>localStorage</code> , and build the auth header.
<code>src/Auth.jsx</code>	The login / register form shown when logged out.
<code>src/App.jsx</code>	Decides: no token → show form; has token → show todos. Adds the token to every fetch.

The login journey, step by step

- 1 App loads. `App.jsx` checks `localStorage` for a token. None found → render `<Auth />` .
- 2 User types email + password, hits Register. `Auth.jsx` sends `POST /register` .
- 3 Server hashes the password, saves the user, signs a token, returns it.
- 4 `Auth.jsx` calls `onAuth(token)` → `App.jsx` saves it to `localStorage` and re-renders.
- 5 Now there is a token → App shows the todo list and fetches `GET /todos` with the token attached.
- 6 Server's `authMiddleware` verifies the token, learns `userId` , returns only that user's todos.
- 7 Refresh the page? Token is still in `localStorage` → stay logged in. Log out → token cleared → back to the form.

5. Per-user data (authorization)

Every todo is stamped with the owner's id, and every query filters by the logged-in user. That's why two accounts never see each other's todos.

```
// create: stamp the owner
todos.push({ id, title, done: false, userId: req.userId })

// read: only mine
todos.filter(t => t.userId === req.userId)
```

6. Run it

```
npm run server # terminal 1 – backend on :3001
npm run dev    # terminal 2 – React (Vite)
```

Open the Vite URL → Register → add todos → Log out → refresh stays logged out.

7. Kept simple on purpose

This project (learning)	Real production app
Users in an in-memory array (lost on restart)	A database (Postgres, SQLite...)
Token in <code>localStorage</code>	httpOnly cookie (safer vs. XSS)
Secret falls back to a dev default	Strong secret from an env var, never in code

Security note: the JWT secret signs every token. Anyone who knows it can forge a token and log in as anyone. Keep it in an env var, out of git: `JWT_SECRET=$(openssl rand -hex 32) npm run server`

Mental shortcut: password = the key you use *once* to get in. Token = the wristband you wear after, shown at every door until it expires.